

Single-Cell RNA-seq Analysis Pipeline

From Raw Reads to Differential Expression and GO Enrichment

2026-03-31

Contents

1 Overview	3
2 Requirements	4
2.1 Hardware	4
2.2 Docker installation	4
2.2.1 Linux (Ubuntu/Debian)	4
2.2.2 macOS / Windows	4
2.3 Pulling the container	4
2.4 Starting an interactive session	4
3 Step 1: Reference Genome Preparation	5
3.1 Downloading the genome and annotation	5
3.2 Filtering to protein-coding genes	5
3.3 Building the CellRanger reference index	5
4 Step 2: Read Quantification (CellRanger)	6
4.1 FASTQ file naming convention	6
4.2 Running cellranger count	6
5 Step 3: Ambient RNA Removal (CellBender)	7
6 Step 4: Pipeline Configuration	8
7 Step 5: Quality Control	9
7.1 Pre-filter assessment	9
7.2 Filtering and doublet detection	9
8 Step 6: Data Merging, Normalisation, and Preprocessing	10
9 Step 7: Batch Correction (Harmony) and Dimension Selection	11
9.1 Elbow plot	11
10 Step 8: Clustering Resolution Selection (Clustree)	12
11 Step 9: Final Clustering	13
12 Step 10: Cell Count per Cluster	14

13 Step 11: Global Pseudobulk Replicate Correlation	15
14 Step 12: Marker Gene Identification and Cell-Type Annotation	16
14.1 Marker genes and literature-based annotation	16
14.2 Reference-based annotation	16
14.3 Annotated clustree	16
15 Step 13: Gene Expression Visualisation	17
16 Step 14: Cell-Type Grouping and Manual Curation	18
16.1 Grouping fine annotations	18
16.2 Manual subclustering and curation	18
17 Step 15: Marker Gene Dot Plot	20
18 Step 16: Export to Python (h5ad)	21
19 Step 17: Pseudobulk per Cell Type and DESeq2 Differential Expression	22
19.1 Pseudobulk aggregation	22
19.2 DESeq2 analysis	22
19.3 Volcano plots and differential heatmaps	23
20 Step 18: GO Enrichment Analysis	24
21 Adapting the Pipeline to Other Organisms	25
22 Output File Summary	26

1 Overview

This document describes a complete, reproducible computational pipeline for analysing single-cell RNA sequencing (scRNA-seq) data generated on the 10x Genomics Chromium platform. The workflow starts from raw FASTQ files, aligns reads to a reference genome, removes ambient RNA contamination, performs quality control, integrates multiple samples by correcting batch effects, clusters cells, assigns cell-type identities, and finally performs pseudobulk differential expression and gene ontology (GO) enrichment analysis.

All software runs inside a Docker container (`mvergara19/scrnaseq_docker:latest`), which packages R 4.5, Python 3, Seurat 5, harmony, DESeq2, CellRanger 9.0.1, CellBender, and all dependencies into a single reproducible environment. No local R or Python installation is needed beyond Docker itself.

The pipeline is **organism-agnostic**. The examples throughout this document use *Arabidopsis thaliana* under three nitrogen conditions (0N, 0.5N, and 5N), but adapting it to human, mouse, or any other species requires changing only three parameters: the mitochondrial gene prefix, the GO annotation database, and the key type.

2 Requirements

2.1 Hardware

Resource	Minimum	Recommended
RAM	16 GB	64 GB (datasets > 30,000 cells)
Storage	50 GB free	200 GB
CPU	4 cores	24 cores (for CellRanger)

2.2 Docker installation

2.2.1 Linux (Ubuntu/Debian)

```
sudo apt-get update && sudo apt-get install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER    # log out and back in after this
```

2.2.2 macOS / Windows

Download Docker Desktop from <https://www.docker.com/products/docker-desktop/> and follow the installer. On Windows, WSL 2 is required and the installer guides the setup.

2.3 Pulling the container

```
docker pull mvergara19/scrnaseq_docker:latest
```

2.4 Starting an interactive session

```
docker run --rm -it -v $(pwd):/workspace mvergara19/scrnaseq_docker:latest bash
```

The `-v` flag mounts the current directory into `/workspace` inside the container. All input data and output files should live under this directory.

3 Step 1: Reference Genome Preparation

Before quantifying gene expression, a species-specific reference must be built from two files: a genome sequence (`.fa`) and a gene annotation (`.gtf`). This step is performed once per organism.

3.1 Downloading the genome and annotation

For *Arabidopsis thaliana* (Ensembl Plants release 53):

```
wget http://ftp.ensemblgenomes.org/pub/plants/release-53/fasta/arabidopsis_thaliana/dna/Arabidopsis_thaliana.dna.toplevel.fa
wget http://ftp.ebi.ac.uk/ensemblgenomes/pub/release-53/plants/gtf/arabidopsis_thaliana/Arabidopsis_thaliana.TAIR10.53.filtrado.gtf
gzip -d Arabidopsis_thaliana.*
```

For **human** or **mouse**, pre-built CellRanger references are available at the 10x Genomics support website.

3.2 Filtering to protein-coding genes

CellRanger recommends restricting the annotation to protein-coding genes to reduce multi-mapping and improve accuracy:

```
cellranger mkgtf \
  Arabidopsis_thaliana.TAIR10.53.gtf \
  Arabidopsis_thaliana.TAIR10.53.filtrado.gtf \
  --attribute=gene_biotype:protein_coding
```

3.3 Building the CellRanger reference index

```
cellranger mkref \
  --genome=Arabidopsis_thaliana.TAIR10.53.transcriptome \
  --fasta=Arabidopsis_thaliana.TAIR10.dna.toplevel.fa \
  --genes=Arabidopsis_thaliana.TAIR10.53.filtrado.gtf
```

This generates an indexed reference directory used by all subsequent `cellranger count` calls. Building the reference typically takes 20–40 minutes.

4 Step 2: Read Quantification (CellRanger)

CellRanger aligns cDNA reads to the reference genome using STAR, corrects cell barcodes, collapses UMIs, and produces a filtered count matrix in which each column is a cell and each row is a gene.

4.1 FASTQ file naming convention

Each 10x Genomics library produces three FASTQ files per sequencing lane:

Suffix	Content
*_I1_001.fastq	Sample index (8 bp)
*_R1_001.fastq	Cell barcode (16 bp) + UMI (12 bp)
*_R2_001.fastq	cDNA read (the actual transcript sequence)

For example, a library with accession SRR8485805 would have files named:

```
SRR8485805_S1_L001_I1_001.fastq
SRR8485805_S1_L001_R1_001.fastq
SRR8485805_S1_L001_R2_001.fastq
```

4.2 Running cellranger count

```
cellranger count \
  --localcores=24 \
  --id=run_count \
  --fastqs=/workspace/fastqs \
  --sample=SRR8485805 \
  --transcriptome=/workspace/Arabidopsis_thaliana.TAIR10.53.transcriptome
```

Replace `--sample` with the filename prefix of your FASTQ files, and `--transcriptome` with the path to the reference built in Step 1. This step takes several hours per sample. The key output is `run_count/outs/raw_feature_bc_matrix.h5` (required by CellBender) and `run_count/outs/filtered_feature_bc_matrix/` (MEX format, for direct use without CellBender).

5 Step 3: Ambient RNA Removal (CellBender)

During library preparation, lysed cells release mRNA into the cell suspension. This ambient RNA is captured by empty droplets and inflates the counts of certain genes across all cells. CellBender uses a deep generative model to estimate the ambient RNA profile and subtract it from each cell's count vector.

```
cellbender remove-background \  
  --input /workspace/raw_feature_bc_matrix.h5 \  
  --output /workspace/Sample_cellbender_output.h5
```

The corrected output (`Sample_cellbender_output_filtered.h5`) is the recommended input for all downstream R analyses. Running CellBender on a GPU significantly reduces computation time; to enable GPU support, launch the container with `--gpus all` and ensure the NVIDIA Container Toolkit is installed on the host.

6 Step 4: Pipeline Configuration

All experiment-specific parameters are defined once at the top of `paperrrr.R`. The `samples` list points to each CellBender output file and assigns a label and condition to each sample. The `colors` vector maps sample labels to colours used in all plots. The `output_dir` sets the destination for all output files.

```
source("load_libraries.R")
source("custom_seurat.R")
source("ScRNA_Analysis_Functions.R")

setwd("/path/to/your/project/")

samples <- list(
  list(file = "cellbender/Sample_0N_cellbender_filtered.h5",
        label = "0N", condition = "0N"),
  list(file = "cellbender/Sample_05N_R1_cellbender_filtered.h5",
        label = "0.5N_R1", condition = "0.5N"),
  list(file = "cellbender/Sample_5N_R1_cellbender_filtered.h5",
        label = "5N_R1", condition = "5N")
)

colors <- c("0N" = "#66c2a5", "0.5N_R1" = "#fc8d62", "5N_R1" = "#8da0cb")

output_dir <- "results"
resolutions_test <- c(0.15, 0.30, 0.50, 0.8, 1.0)
marcadores <- read.table("biblio_marks.txt", header = TRUE,
                        sep = "\t", quote = "")
```


7 Step 5: Quality Control

Quality control (QC) identifies and removes low-quality cells before downstream analysis. Three metrics are computed per cell:

- **nFeature_RNA** — number of distinct genes detected. Very low values indicate empty droplets; unusually high values may indicate doublets.
- **nCount_RNA** — total UMI count; correlated with sequencing depth.
- **percent.mt** / **percent.cp** — percentage of reads mapping to mitochondrial (or chloroplast, for plants) genes. Elevated values indicate damaged cells that have lost cytoplasmic mRNA.

7.1 Pre-filter assessment

Samples are loaded and mitochondrial/chloroplast genes are annotated. Violin plots are generated before any filtering to guide threshold selection.

```
# Load and annotate (no filtering yet)
# mt_pattern / cp_pattern: Arabidopsis = "^ATMG" / "^ATCG"
#                               Human = "^MT-" / NULL   | Mouse = "^mt-" / NULL
seurat_list_raw <- lapply(samples, load_sample,
                          mt_pattern = "^ATMG", cp_pattern = "^ATCG")
names(seurat_list_raw) <- sapply(samples, `[`, "label")

plots_pre <- imap(seurat_list_raw, ~ plot_qc_violin_grid(.x, .y, colors[[.y]]))
save_qc(plots_pre, "qc_prefilter.pdf")
```

7.2 Filtering and doublet detection

Based on the pre-filter distributions, thresholds are applied. DoubletFinder is then run per sample to identify and remove co-encapsulated cell pairs.

```
# Filter cells and run DoubletFinder per sample
seurat_list <- lapply(seurat_list_raw, filter_sample,
                     min_features = 200, max_mt = 5)
names(seurat_list) <- sapply(samples, `[`, "label")

plots_post <- imap(seurat_list, ~ plot_qc_violin_grid(.x, .y, colors[[.y]]))
save_qc(plots_post, "qc_postfilter.pdf")
```

8 Step 6: Data Merging, Normalisation, and Preprocessing

After quality control, all samples are merged into a single Seurat object and preprocessed through normalisation, variable feature selection, scaling, PCA, and UMAP.

```
pbmc_harmony <- reduce(seurat_list, merge) %>%  
  NormalizeData(verbose = FALSE) %>%  
  FindVariableFeatures(nfeatures = 2000, verbose = FALSE) %>%  
  ScaleData(verbose = FALSE) %>%  
  RunPCA(npcs = 30, verbose = FALSE) %>%  
  RunUMAP(reduction = "pca", dims = 1:30, verbose = FALSE)  
  
pbmc_harmony$orig.ident_uni <- pbmc_harmony$condition  
  
save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),  
  "umap_preharmony.pdf")
```

The UMAP below shows all cells before batch correction, coloured by sample. Separation of cells by sample confirms the presence of batch effects that must be corrected.

9 Step 7: Batch Correction (Harmony) and Dimension Selection

Harmony iteratively adjusts the PCA embedding to remove sample-specific shifts while preserving biological signal.

```
dims_use <- 1:30
k_param <- 30

pbmc_harmony <- pbmc_harmony %>%
  RunHarmony("orig.ident", plot_convergence = FALSE)
```

9.1 Elbow plot

The number of informative PCA dimensions is assessed using a k-means WSS elbow plot. The optimal value corresponds to the inflection point of the curve.

```
k_range <- 2:40
pca_data <- Embeddings(pbmc_harmony, "pca")[, dims_use]
wss <- sapply(k_range, function(k)
  kmeans(pca_data, centers = k, nstart = 10)$tot.withinss)

elbow_plot <- ggplot(data.frame(k = k_range, wss = wss), aes(k, wss)) +
  geom_line() + geom_point() +
  labs(x = "Number of clusters (k)", y = "Within-cluster sum of squares") +
  theme_minimal()

save_pdf(elbow_plot, "elbow_plot.pdf", w = 8, h = 6)
```

10 Step 8: Clustering Resolution Selection (Clustree)

Clustering is run across a range of resolutions and visualised as a tree to select a stable partition.

```
clu <- pbmc_harmony %>%  
  RunUMAP(reduction = "harmony", dims = dims_use, verbose = FALSE) %>%  
  FindNeighbors(reduction = "harmony", dims = dims_use,  
               k.param = k_param, verbose = FALSE)  
  
for (res in resolutions_test)  
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)  
  
save_pdf(clustree(clu, prefix = "RNA_snn_res."), "clustree.pdf", w = 14, h = 14)
```

11 Step 9: Final Clustering

The chosen resolution is applied for the final clustering and results are visualised on the Harmony-corrected UMAP.

```
cluster_resolution <- 0.3

pbmc_harmony <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = dims_use, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = dims_use,
               k.param = k_param, verbose = FALSE) %>%
  FindClusters(resolution = cluster_resolution, algorithm = 4, verbose = FALSE)

save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
         "umap_postharmony.pdf")
save_pdf(DimPlot(pbmc_harmony, group.by = "seurat_clusters",
               cols = sample(colors(distinct = TRUE),
                             length(unique(pbmc_harmony$seurat_clusters)))),
         "umap_seuratclusters.pdf")
```

12 Step 10: Cell Count per Cluster

Bar charts summarise the composition of each cluster by sample and by cluster identity.

```
Idents(pbmc_harmony) <- "orig.ident"
save_pdf(plot_integrated_clusters(pbmc_harmony), "conteocelulas.pdf",
          w = 14, h = 10)

Idents(pbmc_harmony) <- "seurat_clusters"
save_pdf(plot_integrated_clusters(pbmc_harmony), "conteocelulas_seurat.pdf",
          w = 14, h = 10)
```

13 Step 11: Global Pseudobulk Replicate Correlation

All cells per sample are aggregated into a pseudobulk count vector and Pearson correlations between samples are computed. Replicates from the same condition should cluster together with $r > 0.95$.

```
pseudobulk <- generate_pseudobulk(pbmc_harmony, group_by = "orig.ident")

save_pdf(plot_replicate_correlation(pseudobulk$by_sample),
         "pseudobulk_correlation.pdf", w = 8, h = 8)
```

14 Step 12: Marker Gene Identification and Cell-Type Annotation

14.1 Marker genes and literature-based annotation

Differentially expressed genes per cluster are identified with a Wilcoxon test and cross-referenced against curated cell-type signatures from the literature.

```
markers <- find_markers(pbmc_harmony,
                        output_file = file.path(output_dir, "FindAllMarkers.tsv"))

pbmc_harmony <- annotate_by_markers(pbmc_harmony, markers,
                                   reference_file = file.path(output_dir, "biblio_marks.txt"))
```

14.2 Reference-based annotation

Cell-type labels are transferred from a published reference Seurat object.

```
esp <- readRDS(file.path(output_dir, "reference_seurat.rds"))
pbmc_harmony <- annotate_by_reference(pbmc_harmony,
                                     reference_obj = esp,
                                     reference_col = "annotation")
```

14.3 Annotated clustree

The resolution sweep is repeated overlaying cell-type labels to confirm that the chosen resolution separates biologically distinct populations.

```
Mode <- function(x) { ux <- unique(x); ux[which.max(tabulate(match(x, ux)))] }

clu <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = dims_use, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = dims_use,
               k.param = k_param, verbose = FALSE)

for (res in resolutions_test)
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)

save_pdf(
  clustree(clu, prefix = "RNA_snn_res.",
           node_label = "celltype_reference", node_label_aggr = "Mode"),
  "clustree_annotated.pdf", w = 14, h = 14
)
```


15 Step 13: Gene Expression Visualisation

Individual genes or gene sets can be inspected with violin plots and feature plots, globally or restricted to a single cell type.

```
pbmc_harmony      <- JoinLayers(pbmc_harmony)    # required in Seurat 5 after merge
Idents(pbmc_harmony) <- "celltype_reference"

gene              <- "AT5G26000"                 # replace with your gene of interest
genes_of_interest <- c("AT5G26000", "AT5G54250")
celltype          <- "Guard Cell"
sub_obj           <- subset(pbmc_harmony, idents = celltype)

# Single gene - all cell types
save_vln(VlnPlot(pbmc_harmony, features = gene),    "vln_gene_all.pdf")
save_pdf(FeaturePlot(pbmc_harmony, features = gene), "feature_gene_all.pdf")

# Gene set - all cell types
save_vln(VlnPlot(pbmc_harmony, features = genes_of_interest),
          "vln_geneset_all.pdf", n = length(genes_of_interest))

# Single gene - one cell type
save_vln(VlnPlot(sub_obj, features = gene),         "vln_gene_celltype.pdf")
save_pdf(FeaturePlot(sub_obj, features = gene),      "feature_gene_celltype.pdf")
```

16 Step 14: Cell-Type Grouping and Manual Curation

16.1 Grouping fine annotations

Fine-grained labels are collapsed into broader biological categories via a named character vector.

```
grouping <- c(
  "Companion Cell"    = "Vascular Cell",
  "Cambium"           = "Vascular Cell",
  "Phloem Parenchyma" = "Vascular Cell",
  "Xylem"              = "Vascular Cell",
  "Sieve Element"     = "Vascular Cell",
  "Meristemoid"       = "Stomatal Line"
)

pbmc_harmony$annotation_agrupada <- recode(pbmc_harmony$celltype_reference,
                                           !!!grouping)

save_pdf(DimPlot(pbmc_harmony, group.by = "annotation_agrupada",
                 label = TRUE, repel = TRUE, raster = FALSE),
         "umap_annotated.pdf")
```

16.2 Manual subclustering and curation

Heterogeneous populations are subclustered and reassigned via an explicit mapping table.

```
Idents(pbmc_harmony) <- "annotation_agrupada"

# 1. Subcluster heterogeneous types
meristemoid_umap <- subclustar_tipo(pbmc_harmony, "Stomatal Line")
pavement_cell_umap <- subclustar_tipo(pbmc_harmony, "Pavement Cell")

# 2. Inspect with DimPlot / FeaturePlot (interactive step)

# 3. Reassignment table - fill in after inspection
reassign <- list(
  meristemoid_umap = c(
    "0" = "Stomatal Line", "1" = "Stomatal Line",
    "2" = "Pavement Cell", "3" = "Stomatal Line", "4" = "Stomatal Line"
  ),
  pavement_cell_umap = c(
    "0" = "Pavement Cell", "1" = "Pavement Cell",
    "2" = "Pavement Cell", "3" = "Mesophyll", "4" = "Pavement Cell"
  )
)

# 4. Apply corrections
pbmc_harmony$celltype_reference_curated <- pbmc_harmony$annotation_agrupada
```

```
for (obj_name in names(reassign)) {  
  obj <- get(obj_name)  
  pbmc_harmony$celltype_reference_curated[colnames(obj)] <-  
    reassign[[obj_name]][obj$cluster_subtipo]  
}  
  
save_pdf(DimPlot(pbmc_harmony, group.by = "celltype_reference_curated",  
                label = TRUE, repel = TRUE, raster = FALSE),  
         "umap_curada.pdf")
```

17 Step 15: Marker Gene Dot Plot

A dot plot summarises marker gene expression across all curated cell types. Colour encodes average expression; dot size encodes the fraction of expressing cells.

```
cell_order_dotplot <- c(
  "Pavement Cell", "Stomatal lineage", "Guard Cell", "Mesophyll",
  "Bundle Sheath", "Phloem Parenchyma", "Cambium", "Xylem",
  "Companion Cell", "Hydathode", "Cell Cycle: G1-S", "Cell Cycle: G2-M"
)

hacer_dotplot_marcadores(
  pbmc_harmony, marcadores,
  annot_col      = "celltype_reference_curated",
  cell_order     = cell_order_dotplot,
  clusters_remove = c("Sieve Element", "Myrosin Idioblast"),
  outfile        = file.path(output_dir, "dotplot_marcadores.pdf"),
  width = 20, height = 10
)
```

18 Step 16: Export to Python (h5ad)

The curated Seurat object is exported to AnnData format for downstream Python analysis (scanpy, scFates, palantir).

```
# Export full curated object
exportar_para_scanpy(pbmc_harmony, "results/objs/pbmc_harmony_curated.h5ad")

# Export a single cell type
exportar_para_scanpy(
  subset(pbmc_harmony, subset = celltype_reference_curated == "Guard Cell"),
  "results/objs/GuardCell.h5ad"
)
```

19 Step 17: Pseudobulk per Cell Type and DESeq2 Differential Expression

19.1 Pseudobulk aggregation

Cells are split by curated cell type, pseudo-replicates are assigned, and counts are summed per replicate.

```
celular_subsets <- setNames(  
  lapply(unique(pbmc_harmony$celltype_reference_curated),  
    function(t) subset(pbmc_harmony,  
                        subset = celltype_reference_curated == t)),  
  gsub("^[:alnum:]", "_", unique(pbmc_harmony$celltype_reference_curated))  
)  
  
celular_subsets_replicados <- Filter(Negate(is.null),  
  lapply(celular_subsets,  
    asignar_pseudoreplicados))  
pseudobulk_list <- lapply(celular_subsets_replicados, hacer_pseudobulk)  
  
dir.create("results/pseudobulk_replicas", recursive = TRUE,  
  showWarnings = FALSE)  
for (tipo in names(pseudobulk_list))  
  write.csv(pseudobulk_list[[tipo]],  
    file.path("results/pseudobulk_replicas",  
      paste0("Pseudobulk_", tipo, ".csv")),  
    row.names = TRUE)
```

19.2 DESeq2 analysis

All pairwise condition comparisons are run automatically. Edit the `comparaciones` list to match your condition names.

```
comparaciones <- list(  
  list(conds = c("0.5N", "5N"), tag = "05_5"),  
  list(conds = c("0N", "5N"), tag = "0_5"),  
  list(conds = c("0N", "0.5N"), tag = "0_05")  
)  
  
for (tag in sapply(comparaciones, `[`, "tag"))  
  dir.create(file.path("results/deseq2", tag), recursive = TRUE,  
    showWarnings = FALSE)  
  
for (tipo in names(pseudobulk_list))  
  correr_deseq2(as.matrix(pseudobulk_list[[tipo]]), comparaciones,  
    output_dir = "results/deseq2")
```

19.3 Volcano plots and differential heatmaps

```
dir.create("results/volcano_plots", showWarnings = FALSE)

for (comp in comparaciones) {
  tag      <- comp$tag
  csv_files <- list.files(file.path("results/deseq2", tag),
                        pattern = "\\*.csv$", full.names = TRUE)
  if (!length(csv_files)) next

  pdf(file.path("results/volcano_plots",
                paste0("VolcanoPlots_", tag, ".pdf")), width = 12, height = 6)
  plots <- list()
  for (f in csv_files) {
    plots <- c(plots, list(hacer_volcano(f)))
    if (length(plots) == 2) { grid.arrange(grobs = plots, ncol = 2); plots <- list() }
  }
  if (length(plots)) grid.arrange(grobs = plots, ncol = 1)
  dev.off()
}

for (comp in comparaciones) {
  tag      <- comp$tag
  diff_dir <- file.path("results/diff", tag)
  dir.create(diff_dir, recursive = TRUE, showWarnings = FALSE)

  listas      <- lapply(list.files(file.path("results/deseq2", tag),
                                    pattern = "\\*.csv$", full.names = TRUE),
                        procesar_deseq2_resultado, output_dir = diff_dir)
  tabla_logfc <- Reduce(function(x, y) full_join(x, y, by = "gene_id"),
                        lapply(listas, `[`, "logfc"))
  matriz      <- as.matrix(column_to_rownames(tabla_logfc, "gene_id"))
  matriz[is.na(matriz)] <- 0

  if (nrow(matriz) > 1) {
    pdf(file.path(diff_dir, paste0("heatmap_", tag, ".pdf")),
        width = 14, height = 18)
    tryCatch(hacer_heatmap(matriz),
             error = function(e) message("Heatmap error: ", e$message))
    dev.off()
  }
}
```

20 Step 18: GO Enrichment Analysis

GO enrichment is performed per comparison using `clusterProfiler`. Both full and simplified term sets are tested and visualised as dot plots.

```
# Organism: Arabidopsis = org.At.tair.db / "TAIR"
#           Human       = org.Hs.eg.db   / "ENSEMBL"
#           Mouse       = org.Mm.eg.db   / "ENSEMBL"
orgdb      <- org.At.tair.db
keytype    <- "TAIR"
universo   <- keys(orgdb, keytype = keytype)
espacio     <- "BP"      # "BP" | "MF" | "CC"
qval       <- 0.05
nivel_poda <- 6

dir.create("results/Enrichment", showWarnings = FALSE)

for (comp in comparaciones) {
  tag      <- comp$tag
  enr_dir  <- file.path("results/Enrichment", tag)
  tabla_path <- file.path("results/diff", tag, "tabla_diferenciales.tsv")
  dir.create(enr_dir, recursive = TRUE, showWarnings = FALSE)
  if (!file.exists(tabla_path)) next

  tabla      <- read.table(tabla_path, header = TRUE, row.names = 1, sep = "\t")
  go_total   <- correr_enriquecimiento_go(tabla, universo, espacio,
                                           orgdb = orgdb, keytype = keytype, simplificar = FALSE,
                                           output_dir = enr_dir)
  go_simple  <- correr_enriquecimiento_go(tabla, universo, espacio,
                                           orgdb = orgdb, keytype = keytype, simplificar = TRUE,
                                           output_dir = enr_dir)

  go_total_podado <- podar_go(go_total, nivel_poda, espacio, qval,
                              simplificar = FALSE)
  go_simple_podado <- podar_go(go_simple, nivel_poda, espacio, qval,
                               simplificar = TRUE)

  pdf(file.path(enr_dir, paste0("GO_enrichment_", tag, ".pdf")),
      width = 18, height = 18)
  tryCatch({
    print(graficar_go_balones(go_total))
    print(graficar_go_balones(go_simple))
    print(graficar_go_balones(go_total_podado))
    print(graficar_go_balones(go_simple_podado))
  }, error = function(e) message("GO plot error: ", e$message))
  dev.off()
}
```


21 Adapting the Pipeline to Other Organisms

Three parameters in `paperrr.R` control the organism-specific parts of the analysis.

Mitochondrial / chloroplast gene patterns (QC section):

Organism	mt_pattern	cp_pattern
<i>Arabidopsis thaliana</i>	"^ATMG"	"^ATCG"
Human	"^MT-"	NULL
Mouse	"^mt-"	NULL

GO enrichment database (GO ENRICHMENT section):

```
# Arabidopsis
orgdb <- org.At.tair.db; keytype <- "TAIR"

# Human
orgdb <- org.Hs.eg.db; keytype <- "ENSEMBL"

# Mouse
orgdb <- org.Mm.eg.db; keytype <- "ENSEMBL"
```

DESeq2 comparisons — edit the `comparaciones` list to match your condition names:

```
comparaciones <- list(
  list(conds = c("Control", "Treatment"), tag = "ctrl_vs_treat"),
  list(conds = c("Control", "HighDose"), tag = "ctrl_vs_high")
)
```

22 Output File Summary

File	Description
qc_prefilter.pdf	QC violin plots before filtering
qc_postfilter.pdf	QC violin plots after filtering
umap_preharmony.pdf	UMAP before batch correction
elbow_plot.pdf	K-means WSS elbow plot
clustree.pdf	Resolution sweep visualisation
umap_postharmony.pdf	UMAP after Harmony, by sample
umap_seuratclusters.pdf	UMAP coloured by cluster
conteocelulas.pdf	Cell count by sample per cluster
conteocelulas_seurat.pdf	Cell count by cluster
pseudobulk_correlation.pdf	Replicate Pearson correlation heatmap
FindAllMarkers.tsv	Marker gene table
clustree_annotated.pdf	Resolution sweep with cell-type labels
umap_annotated.pdf	UMAP with grouped cell-type labels
umap_curada.pdf	UMAP after manual curation
dotplot_marcadores.pdf	Marker gene dot plot
results/pseudobulk_replicas/*.csv	Pseudobulk count matrices per cell type
results/deseq2/<tag>/*.csv	DESeq2 results per cell type and comparison
results/volcano_plots/*.pdf	Volcano plots per comparison
results/diff/<tag>/heatmap_*.pdf	Log2FC heatmaps
results/Enrichment/<tag>/*.pdf	GO enrichment dot plots
results/objs/*.h5ad	Exported AnnData objects